

Interview Questions & Brief Answers - Java Developer

Core Java & Java 8 Features

- **Java 8 Features: What are they? Why were functional interfaces and Lambda expressions added?**
 - **Brief Answer:** Key features include **Lambda Expressions, Functional Interfaces, Stream API, Optional class, Default Methods in Interfaces, and java.time API**. They were added to **support functional programming paradigms**, enabling more concise, readable, and parallelizable code, especially for collections.
- **Functional Interfaces: What is a functional interface? What are built-in functional interfaces (e.g., Supplier, Consumer)? What are default methods in functional interfaces? What happens when you override default methods?**
 - **Brief Answer:** A **functional interface** is an interface with **exactly one abstract method** (can have multiple default/static methods). Built-in examples: Predicate (test), Consumer (accept), Function (apply), Supplier (get), BiFunction, etc. **Default methods** provide a way to add new methods to interfaces without breaking existing implementations. When overridden, the **subclass's/implementing class's version takes precedence**.
- **Optional Class: What is the Optional class? What are its built-in methods (of, ofNullable, empty)? Explain scenario-based examples.**
 - **Brief Answer:** Optional is a **container object used to contain not-null objects**. It's used to **avoid NullPointerExceptions** and provide a clear way to represent the absence of a value.
 - Optional.of(value): Creates an Optional with the non-null value. Throws NPE if value is null.
 - Optional.ofNullable(value): Creates an Optional with the specified value, or an empty Optional if the value is null.
 - Optional.empty(): Returns an empty Optional instance.
 - **Scenario:** Instead of returning null from a method that might not find a result, return Optional.empty(). Callers can then use isPresent(), orElse(), map(), etc., to handle the absence gracefully.
- **Collections:**
 - **HashMap & ConcurrentHashMap: How do they work internally? Have you used ConcurrentHashMap? How many null keys and values can HashMap store? Explain collision in HashMap.**
 - **Brief Answer:**
 - **HashMap:** Uses an **array of Node objects (or Entry before Java 8)**. Each Node stores key, value, hash, and a pointer to the next node. **Collisions** (different keys producing the same hash bucket) are handled by **linked lists (or balanced trees in Java 8+ for high**

- collision rates) at each array index. Allows **one null key and multiple null values**.
 - **ConcurrentHashMap**: A thread-safe, non-blocking map. Uses **segmentation (Java 7) or a Node array with synchronized on nodes/hash bins (Java 8)**, allowing concurrent reads and writes without blocking the entire map. Yes, used for multi-threaded scenarios.
- **ArrayList vs. LinkedList: What's the difference? Which is faster for element removal? Which is better if your pointer is at the last element? Provide custom implementations.**
 - **Brief Answer:**
 - **ArrayList: Resizable array** implementation. Fast random access ($\text{get}(\text{index}) - O(1)$), slow insertions/deletions in the middle ($O(n)$). Faster for removal if you know the index directly ($O(1)$ if last element, $O(n)$ otherwise).
 - **LinkedList: Doubly-linked list** implementation. Fast insertions/deletions at any position ($O(1)$ once iterator is at position), slow random access ($O(n)$). Faster for removal if you have an iterator pointing to the element ($O(1)$).
 - **Pointer at last element: LinkedList** ($O(1)$ to remove the last element if you have a reference, ArrayList might be $O(1)$ too depending on implementation).
 - *Custom implementations are too extensive for a brief answer here, focus on describing the underlying data structures.*
- **Other Collections:** HashSet internal implementation, TreeMap, LinkedHashMap. Which collections are preferable for async operations?
 - **Brief Answer:**
 - **HashSet**: Internally uses a HashMap where elements are stored as keys and a dummy Object is stored as values.
 - **TreeMap**: Stores elements in a **sorted order** (natural ordering or custom Comparator). Implemented using a **Red-Black tree**.
 - **LinkedHashMap**: Maintains insertion order (or access order). Uses a **doubly-linked list** running through its entries.
 - **Async Operations: ConcurrentHashMap**, CopyOnWriteArrayList, BlockingQueue implementations (LinkedBlockingQueue, ArrayBlockingQueue).
- **List Operations:** How to remove duplicates from a list (without using Collections or Streams, using Java 8 streams)? How to convert a list to a map? How to convert a map to a list?
 - **Brief Answer:**
 - **Remove Duplicates (no Collections/Streams)**: Iterate with nested loops, adding unique elements to a new list. Or use a Set (e.g., HashSet) implicitly.

- **Remove Duplicates (Java 8 Streams):**
list.stream().distinct().collect(Collectors.toList());
 - **List to Map:** list.stream().collect(Collectors.toMap(keyExtractor, valueExtractor));
 - **Map to List (Keys):** new ArrayList<>(map.keySet());
 - **Map to List (Values):** new ArrayList<>(map.values());
 - **Map to List (Entries):** new ArrayList<>(map.entrySet());
- **Strings: Difference between String, StringBuffer, and StringBuilder. When to use each?**
 - **Brief Answer:**
 - **String: Immutable.** Best for constant string values.
 - **StringBuffer: Mutable, thread-safe (synchronized).** Slower due to synchronization overhead. Use in multi-threaded environments where string modifications are frequent.
 - **StringBuilder: Mutable, not thread-safe.** Faster than StringBuffer. Use in single-threaded environments where string modifications are frequent.
- **OOP Concepts: What are the basics of OOP? Explain abstraction. How does JVM know which child class's method to execute when overriding?**
 - **Brief Answer:**
 - **Basics: Encapsulation** (bundling data and methods), **Inheritance** (inheriting properties/behaviors), **Polymorphism** (one interface, multiple implementations), **Abstraction** (hiding implementation details, showing only essential features).
 - **Abstraction:** Hiding complex implementation details and showing only the necessary functionalities. Achieved via **abstract classes and interfaces**.
 - **JVM:** Uses **dynamic method dispatch (late binding)**. At runtime, the JVM determines the actual type of the object and invokes the overridden method of that specific object's class.
- **Inheritance & Polymorphism: Method overloading vs. method overriding (rules, exceptions in parent/child classes).**
 - **Brief Answer:**
 - **Overloading: Same method name, different method signatures (number/type/order of parameters) within the same class.** Compile-time polymorphism.
 - **Overriding: Same method signature in different classes (parent-child relationship).** Runtime polymorphism.
 - **Rules for Overriding:**
 - Method signature (name, parameters) must be same.
 - Return type must be same or a covariant return type (subclass of parent's return type).
 - Access modifier cannot be more restrictive.
 - Cannot throw new checked exceptions broader than parent's, can throw unchecked, or narrower checked exceptions.

- final or static methods cannot be overridden.
- **final keyword: What is its use? Why do we need it? Can we override a final method? Can member variables of a final class be changed?**
 - **Brief Answer:**
 - **Use:**
 - **final variable:** Value can be assigned only once (constant).
 - **final method:** Cannot be overridden by subclasses.
 - **final class:** Cannot be subclassed (inherited from).
 - **Need:** For immutability, security, and performance optimizations.
 - **Override final method:** No.
 - **Member variables of final class:** Yes, if they are not themselves final. A final class means its state cannot be *extended* by subclassing, but its *instance's internal state* can change unless its fields are also final or immutable.
- **hashCode & equals: Need for overriding hashCode and equals methods.**
 - **Brief Answer:** Essential when storing custom objects in **hash-based collections** like **HashMap, HashSet, Hashtable**.
 - **equals():** Defines logical equality between two objects (e.g., two Person objects are equal if they have the same ID).
 - **hashCode():** Returns an integer hash code for the object.
 - **Contract:** If two objects are equal according to the equals(Object) method, then calling the hashCode method on each of the two objects must produce the same integer result. If you override equals,¹ you *must* override hashCode.
- **Exceptions: Checked vs. unchecked exceptions. How to handle custom exceptions without ControllerAdvice? Throw vs. Throws. Fail-fast vs. fail-safe iterators.**
 - **Brief Answer:**
 - **Checked:** Must be caught or declared (IOException, SQLException). Compiler forces handling.
 - **Unchecked (Runtime):** Don't need to be caught or declared (NullPointerException, ArrayIndexOutOfBoundsException). Indicate programming errors.
 - **Custom Exceptions without ControllerAdvice:** Use try-catch blocks directly in service or controller methods, or implement a custom HandlerExceptionResolver.
 - **throw:** Used to **explicitly throw an exception** instance.
 - **throws:** Used in a **method signature to declare** that the method might throw a certain type of exception.
 - **Fail-fast:** Iterators that immediately throw a ConcurrentModificationException if the underlying collection is structurally modified during iteration (e.g., ArrayList, HashMap iterators).
 - **Fail-safe:** Iterators that don't throw an exception if the underlying

collection is modified during iteration, as they operate on a copy of the collection (e.g., CopyOnWriteArrayList, ConcurrentHashMap iterators).

- **JVM, JRE, JDK: Explain each.**
 - **Brief Answer:**
 - **JDK (Java Development Kit):** Full development environment. Includes JRE + development tools (compiler javac, debugger jdb, etc.). For developers.
 - **JRE (Java Runtime Environment):** Provides libraries and JVM to run Java applications. For end-users to run Java apps.
 - **JVM (Java Virtual Machine):** Abstract machine that enables a computer to run Java programs. It's the component that executes bytecode. "Write once, run anywhere."
- **Serialization: What is Serializable and when to use it? What are marker interfaces?**
 - **Brief Answer:**
 - **Serializable:** A **marker interface** in Java. Implementing it allows an object to be **converted into a byte stream** (serialized) for storage or transmission, and then reconstructed (deserialized).
 - **When to use:** For persistence (storing objects to disk), inter-process communication, or network communication (sending objects over a network).
 - **Marker Interface:** An interface with **no methods or fields**. It "marks" a class as having a special property or behavior (e.g., Serializable, Cloneable, Remote).
- **Reflection API: What is it and how can it be used?**
 - **Brief Answer:** A Java API that allows an executing Java program to **examine or modify its own structure and behavior (fields, methods, constructors) at runtime**. Used in frameworks (Spring, Hibernate), ORM, testing tools, and dynamic code generation.
- **Garbage Collection: Working of Java garbage collection.**
 - **Brief Answer:** An **automatic memory management process** that identifies and reclaims memory occupied by objects that are no longer referenced by the program. It prevents memory leaks. The JVM has different GC algorithms (e.g., Serial, Parallel, G1, ZGC) that typically work in phases (Mark, Sweep, Compact).
- **Primitive Data Types: Difference between int and Integer.**
 - **Brief Answer:**
 - **int:** A **primitive data type**. Stores direct values. Faster, consumes less memory.
 - **Integer:** A **wrapper class** for int. An **object**. Provides utility methods (e.g., parseInt()). Can be null. Used in collections (which store objects). Auto-boxing/unboxing handles conversions.
- **Record class: What is it?**
 - **Brief Answer:** Introduced in Java 16, a record is a **special kind of class in Java** designed for modeling **plain data aggregates**. It automatically generates boilerplate code like equals(), hashCode(), toString(), and canonical constructor,

reducing verbosity for data carrier classes.

- **Optional String with Optional.empty(): What value will get return?**
 - **Brief Answer:** Calling get() on an Optional.empty() will throw a **NoSuchElementException**. This is why isPresent(), orElse(), orElseThrow(), etc., should be used.
 - **Recursion: Factorial program with recursion.**
 - **Brief Answer:** A function that calls itself to solve a problem, breaking it down into smaller, similar subproblems.
 - `public int factorial(int n) { if (n == 0) return 1; return n * factorial(n - 1); }`
-

Spring & Spring Boot

- **IoC Container: Types of IoC containers. Spring IoC container lifecycle management. Benefits of IoC for developers.**
 - **Brief Answer:**
 - **Types:** BeanFactory (basic) and ApplicationContext (advanced, includes more features like AOP, internationalization, event publishing).
 - **Lifecycle:** The container manages the entire lifecycle of a bean: instantiation, populating properties, initialization (e.g., @PostConstruct), use, and destruction (@PreDestroy).
 - **Benefits:** **Loose coupling, easy testing** (mocking dependencies), **reduced boilerplate code, easier maintenance and scalability.**
- **Spring Beans: Bean scope types and purpose (default scope). Spring bean lifecycle.**
 - **Brief Answer:**
 - **Scopes:**
 - **singleton (default):** One instance per Spring IoC container.
 - **prototype:** A new instance every time it's requested.
 - **request, session, application, websocket:** For web applications.
 - **Lifecycle:**
 1. **Instantiation:** Bean object created.
 2. **Populate Properties:** Dependencies injected.
 3. **Bean Name/Factory/Context Aware:** If interfaces implemented, respective methods called.
 4. **BeanPostProcessor (before initialization):** postProcessBeforeInitialization().
 5. **Initialization:** @PostConstruct or afterPropertiesSet() or custom init method.
 6. **BeanPostProcessor (after initialization):** postProcessAfterInitialization().
 7. **In Use:** Bean is ready for use.
 8. **Destroy:** @PreDestroy or destroy() or custom destroy method.

- **Dependency Injection (DI): Ways to implement DI. What annotations are used for DI (@Autowired, @Resource, @Inject)? Difference between @Autowired and @Resource / @Inject.**
 - **Brief Answer:**
 - **Ways: Constructor Injection** (preferred, ensures immutability and all dependencies are present), **Setter Injection** (for optional dependencies), **Field Injection** (least preferred, harder to test).
 - **Annotations:** @Autowired (Spring specific), @Resource (JSR-250 standard), @Inject (JSR-330 standard).
 - **Difference:**
 - **@Autowired:** Spring-specific. Injects by **type** by default. Can use @Qualifier for ambiguity.
 - **@Resource:** JEE standard. Injects by **name** first, then by type.
 - **@Inject:** JSR-330 standard. Injects by **type**. Similar to @Autowired but more generic.
- **Spring Boot Annotations: What's the function of @SpringBootApplication? Explain @Controller vs. @RestController. Explain @EnableAutoConfiguration. What is @Qualifier? What is @Entity and why is it required? What is @Transactional?**
 - **Brief Answer:**
 - **@SpringBootApplication:** A convenience annotation that combines @Configuration, @EnableAutoConfiguration, and @ComponentScan. It² marks the main class of a Spring Boot application.³
 - **@Controller:** Used to define a **Spring MVC controller**, typically returns view names for rendering HTML.
 - **@RestController:** A specialized version of @Controller that adds the @ResponseBody annotation automatically. Means all methods return data directly (e.g., JSON/XML) instead of a view. Used for **RESTful web services**.
 - **@EnableAutoConfiguration:** Tells Spring Boot to **automatically configure the application** based on classpath dependencies (e.g., if H2 is on classpath, it configures an in-memory database).
 - **@Qualifier:** Used with @Autowired to resolve dependency injection ambiguity when multiple beans of the same type exist. Specifies the exact bean name to inject.
 - **@Entity:** A JPA annotation used to mark a class as a **JPA entity**, meaning it maps to a table in a relational database. Required for ORM frameworks like Hibernate to manage persistence.
 - **@Transactional:** Marks a method or class to be run within a **database transaction**. Spring manages the transaction lifecycle (begin, commit, rollback).
- **@Transactional: How does it work? When does @Transactional not work? How to handle IOException with @Transactional (rollback or not)?**
 - **Brief Answer:**

- **How it works:** Spring creates a **proxy** around the annotated method/class. Before the method executes, it starts a transaction; if the method completes successfully, it commits; if an unchecked exception occurs, it rolls back.
 - **When it doesn't work:**
 - Calling a @Transactional method **from another method within the same class** (due to proxying).
 - private or final methods.
 - No proxy created (e.g., AspectJ weaving not enabled).
 - Checked exceptions by default (unless rollbackFor is specified).
 - Database does not support transactions.
 - **IOException:** By default, @Transactional only rolls back for **unchecked exceptions** (RuntimeException and its subclasses). IOException is a **checked exception**, so it **will not trigger a rollback by default**. To roll back for IOException, you must explicitly specify rollbackFor = IOException.class in the annotation: @Transactional(rollbackFor = IOException.class).
- **Spring Data JPA: Table creation using Spring Data JPA. How does Spring JPA handle joins?**
 - **Brief Answer:**
 - **Table Creation:** Spring Data JPA (via Hibernate as the default JPA provider) uses **ddl-auto properties** (e.g., spring.jpa.hibernate.ddl-auto=update or create) in application.properties to automatically generate tables from @Entity classes based on the entity definitions and dialect.
 - **Joins:** Handles joins implicitly based on **entity relationships** defined with annotations like @OneToOne, @OneToMany, @ManyToOne, @ManyToMany. When you query entities, JPA automatically generates the necessary SQL JOIN clauses based on these mappings, often with **lazy loading** by default to optimize performance. You can control eager/lazy fetching and use @Query annotations for explicit JPQL/native SQL joins.
- **Spring AOP: Concepts and annotations (@Before, @After). How does it work in real-time? How to handle if @Aspect fails?**
 - **Brief Answer:**
 - **AOP (Aspect-Oriented Programming):** A programming paradigm that aims to **modularize cross-cutting concerns** (e.g., logging, security, transactions) that affect multiple parts of an application.
 - **Concepts:**
 - **Aspect:** A module that encapsulates a cross-cutting concern (e.g., LoggingAspect).
 - **Join Point:** A specific point in the execution of the program (method call, exception handling).
 - **Advice:** Action taken by an aspect at a particular join point.
 - **Pointcut:** A predicate that matches join points.

- **Weaving:** Process of linking aspects with other application types/objects.
 - **Annotations (Advice Types):**
 - @Before: Runs before a method execution.
 - @After: Runs after a method execution (regardless of success/failure).
 - @AfterReturning: Runs after a method successfully returns.
 - @AfterThrowing: Runs if a method throws an exception.
 - @Around: Surrounds a method call, providing full control.
 - **Real-time:** Used for logging, performance monitoring, security checks, transaction management, etc., by injecting behavior without modifying core business logic.
 - **@Aspect fails:** An exception thrown within the advice itself can break the application flow. You'd typically wrap the advice logic in try-catch blocks within the aspect to handle potential errors in the advice itself, preventing it from cascading.
- **REST API: Rules to implement REST API. CRUD operations. How to handle global exceptions in Spring Boot? Ways to handle exceptions in Spring Boot.**
 - **Brief Answer:**
 - **REST Rules (RESTful principles):**
 - **Client-Server:** Separation of concerns.
 - **Stateless:** No client context stored on the server.
 - **Cacheable:** Responses can be cached.
 - **Uniform Interface:** Consistent way of interacting (resources, URIs, standard HTTP methods, HATEOAS).
 - **Layered System:** Components can be deployed in hierarchies.
 - **CRUD Operations & HTTP Methods:**
 - Create: POST
 - Read: GET
 - Update: PUT (full update, idempotent) or PATCH (partial update)
 - Delete: DELETE
 - **Global Exception Handling (Spring Boot):** Use @ControllerAdvice combined with @ExceptionHandler.
 - **Ways to handle exceptions:**
 - **try-catch blocks:** For local handling.
 - **@ExceptionHandler:** Method-level or class-level (within @Controller).
 - **@ControllerAdvice / @RestControllerAdvice:** Global handling across multiple controllers.
 - **ResponseStatusException:** For simple HTTP status code exceptions.
- **Spring Actuator: What is it and how is it used?**
 - **Brief Answer:** Provides **production-ready features** to monitor and manage your Spring Boot application. It exposes endpoints (e.g., /health, /info, /metrics, /loggers) that provide insights into the application's state, health, environment,

and various metrics. Used for monitoring, alerting, and operational insights.

- **Spring Profiles: Have you used Spring profiles?**
 - **Brief Answer:** Yes. Spring Profiles allow you to **register different beans for different environments** (e.g., dev, test, prod). You can activate specific profiles using `spring.profiles.active` property or JVM arguments, which helps manage configuration and beans seamlessly across environments.
- **Spring Boot Performance: How to identify a slow Spring Boot app? What needs to be done to improve performance?**
 - **Brief Answer:**
 - **Identify:** Use Spring Actuator metrics, profiling tools (JVisualVM, JProfiler, YourKit), logging (slow queries, long-running methods), monitoring tools (Prometheus, Grafana, ELK stack).
 - **Improve:**
 - **Database optimization:** Indexing, efficient queries, connection pooling.
 - **Caching:** Spring Cache, Redis, Ehcache.
 - **Asynchronous processing:** `@Async` methods, messaging queues.
 - **Resource management:** Thread pools, connection pools.
 - **Code optimization:** Algorithmic improvements, avoiding unnecessary loops/objects.
 - **JVM Tuning:** Heap size, GC tuning.
 - **Load Balancing and Scaling.**
- **RestTemplate: How to connect REST API using RestTemplate?**
 - **Brief Answer:** RestTemplate (now largely superseded by WebClient in Spring WebFlux for reactive applications) is a synchronous client for making HTTP requests to consume RESTful web services. You'd inject RestTemplate and use methods like `getForObject()`, `postForEntity()`, `exchange()` to interact with APIs. Example: `restTemplate.getForObject("http://api.example.com/data", MyResponseClass.class);`
- **Default Connection Pool: What is the default connection pool in Spring Boot?**
 - **Brief Answer:** Starting with Spring Boot 2.0, the default connection pool is **HikariCP** due to its superior performance and reliability.
- **@Conditional annotation: What is it?**
 - **Brief Answer:** Allows you to **conditionally register beans or configuration classes** based on certain conditions (e.g., presence of a class, specific property value, bean definition). Used to make configurations more flexible and environment-aware.
- **Proxy in Spring Boot: What is it? Dynamic proxy and its limitations.**
 - **Brief Answer:** Spring uses **proxies (JDK dynamic proxies or CGLIB proxies)** extensively for AOP, `@Transactional`, `@Async`, etc. A proxy is an object that wraps another object (the target) and intercepts method calls to it, adding extra behavior (like transaction management) before/after calling the actual method.
 - **Dynamic Proxy:** Created at runtime using Java's `java.lang.reflect.Proxy` for

interfaces.

- **Limitations:**

- **JDK Dynamic Proxies:** Only for interfaces.
- **CGLIB Proxies:** Can proxy classes, but requires the class and methods not to be final.
- **Self-invocation:** @Transactional or @Async methods called from within the same class instance will bypass the proxy and thus the AOP advice.

- **Manual Database Configuration: How to manually configure the database in Spring Boot?**

- **Brief Answer:** You can manually configure by defining a DataSource bean (e.g., HikariDataSource, DriverManagerDataSource) in a @Configuration class, specifying properties like URL, username, password, and driver class. You might also need to configure a LocalContainerEntityManagerFactoryBean for JPA.

- **Spring Security: JWT and OAuth. How to access entities using security? How is security maintained in your application? How to secure an API? What types of authentication have you managed?**

- **Brief Answer:**

- **JWT (JSON Web Tokens):** A compact, URL-safe means of representing claims to be transferred between two parties. Used for stateless authentication.
- **OAuth (Open Authorization):** An open standard for access delegation, commonly used for delegated authorization (e.g., "Login with Google").
- **Accessing Entities/Securing API:** Spring Security uses **filters** to intercept requests. You define **authentication providers** (e.g., DaoAuthenticationProvider for username/password, JWT filters) and **authorization rules** (@PreAuthorize, @Secured, URL-based rules in SecurityFilterChain).
- **Security Maintenance:** Regular dependency updates, secure coding practices, input validation, output encoding, managing secrets, using secure protocols (HTTPS), logging security events, and regular security audits.
- **Authentication Types:** Token-based (JWT, OAuth), Basic Auth, Form-based, LDAP, SAML.

- **Spring Batch: Do you know Spring Batch?**

- **Brief Answer:** Yes. Spring Batch is a **lightweight, comprehensive framework for robust batch applications**. It provides reusable functions essential for processing large volumes of records, including logging, transaction management, restartability, skip/retry logic, and partitioning. Ideal for ETL processes, data migration, and periodic tasks.
-

Microservices

- **Microservices Architecture: What and why microservices? Problems in microservices architecture. Main pointers of microservices.**
 - **Brief Answer:**
 - **What/Why:** An architectural style where an application is structured as a **collection of loosely coupled, independently deployable, small services** that communicate over a network (e.g., HTTP APIs, message queues).
 - **Benefits (Why): Scalability** (services can scale independently), **Resilience** (failure in one doesn't bring down all), **Technology Diversity, Independent Deployment, Faster Development Cycles.**
 - **Problems:** Increased **operational complexity** (deployment, monitoring, debugging across services), **distributed data management, network latency, transaction management across services, security.**
 - **Main Pointers:** Small, independent, loosely coupled, autonomous teams, domain-driven design, API-driven communication.
- **Communication: How do microservices communicate with each other (synchronous vs. asynchronous)? Ways to achieve synchronous/asynchronous communication. Drawbacks of each.**
 - **Brief Answer:**
 - **Synchronous: HTTP/REST (e.g., RestTemplate, WebClient), gRPC.**
 - **Drawbacks:** Tightly coupled (sender waits for response), network latency, service availability dependency, cascading failures.
 - **Asynchronous: Message Queues (e.g., Kafka, RabbitMQ, ActiveMQ), Event Buses.**
 - **Ways to achieve:** Publish-subscribe models.
 - **Drawbacks:** Increased complexity (eventual consistency, message ordering, message processing guarantees), debugging can be harder.
- **Design Patterns:**
 - **Circuit Breaker: What is it? How does fault tolerance happen using Hystrix?**
 - **Brief Answer:** A design pattern to **prevent cascading failures** in distributed systems. If a service call repeatedly fails, the circuit breaker "trips" (opens), preventing further calls to the failing service and returning a fallback response immediately, giving the failing service time to recover. **Hystrix (now deprecated, often replaced by Resilience4j)** is a library that implements the circuit breaker pattern, providing fallback methods, thread isolation, and metrics.
 - **Saga Pattern: Explain the Saga pattern.**
 - **Brief Answer:** A design pattern for **managing distributed transactions** (long-running business processes) that span multiple services, ensuring data consistency. Instead of a single atomic transaction, a Saga is a sequence of local transactions, where each local transaction updates data

within its service and publishes an event that triggers the next local transaction. If a step fails, compensation transactions are executed to undo the previous steps.

- **API Gateway: Explain the working of an API Gateway if used.**
 - **Brief Answer:** A single entry point for clients to access microservices. It handles concerns like **routing requests** to appropriate services, **authentication/authorization**, **rate limiting**, **caching**, **load balancing**, **request/response transformation**, and **logging**. It decouples clients from individual microservices.
- **CQRS Pattern: What is it?**
 - **Brief Answer: Command Query Responsibility Segregation.** A pattern that separates the **read model (query operations)** from the **write model (command operations)** for a data store. This allows for independent optimization, scaling, and data representation for reading and writing, leading to better performance and flexibility for complex domains.
- **Eureka Server: Explain how Eureka Server works in detail and how microservices register and communicate using Eureka.**
 - **Brief Answer: Eureka Server** acts as a **service registry** in a Netflix OSS (or Spring Cloud) ecosystem.
 - **Registration:** Microservices (clients) **register themselves** with the Eureka Server on startup, sending their hostname, IP address, and port.
 - **Heartbeats:** Clients send **heartbeats** to the server to renew their leases and indicate they are still alive.
 - **Discovery:** Other microservices (clients) **query the Eureka Server** to discover the locations (IPs/ports) of other services they need to communicate with.
 - **Communication:** Once a client has the service location, it can directly communicate with the target service (peer-to-peer). Eureka is for discovery, not for proxying communication.
- **Scaling: How can you scale microservices using Kubernetes?**
 - **Brief Answer:** Kubernetes provides robust scaling capabilities:
 - **Horizontal Pod Autoscaler (HPA):** Automatically scales the number of pods up or down based on CPU utilization or custom metrics.
 - **Vertical Pod Autoscaler (VPA):** Recommends or sets CPU/memory requests/limits for pods.
 - **ReplicaSets/Deployments:** Define desired number of replicas for a service, Kubernetes ensures that many pods are running.
 - **Load Balancing:** Services automatically distribute traffic across healthy pods.
 - **Node Autoscaling:** Can scale the underlying cluster nodes based on resource demand.
- **Monolithic vs. Microservices: Main points to consider when migrating from monolithic to microservices.**

- **Brief Answer:**
 - **Domain Decomposition:** How to break down the monolithic application into meaningful, independent services (bounded contexts).
 - **Data Migration & Consistency:** Managing data across distributed services (e.g., shared database vs. database per service, eventual consistency).
 - **Communication Strategy:** Synchronous vs. asynchronous, API Gateway design.
 - **DevOps & CI/CD:** Increased complexity, need for automation, containerization (Docker, Kubernetes).
 - **Observability:** Distributed logging, tracing, monitoring.
 - **Team Structure:** Aligning teams with service ownership.
-

Databases & SQL

- **SQL Queries:**
 - **How to design a REST API to get data from an Employee table?**
 - **Brief Answer:** Design endpoints like /employees (GET for all), /employees/{id} (GET for specific), /employees (POST for create), /employees/{id} (PUT/PATCH for update), /employees/{id} (DELETE for delete). Use Spring Boot with Spring Data JPA and a @RestController.
 - **SQL question: Employee table with 5 duplicate rows, delete 4 and keep 1.**
 - **Brief Answer:**

```
SQL
DELETE FROM Employee
WHERE ROWID NOT IN (
    SELECT MIN(ROWID)
    FROM Employee
    GROUP BY column1, column2, ... -- Group by all columns that define a
"duplicate"
);
```

*(Using ROWID (Oracle) or CTID (PostgreSQL) is efficient. For SQL Server/MySQL, use DELETE FROM Employee WHERE Id NOT IN (SELECT MIN(Id) FROM Employee GROUP BY ...) or WITH CTE AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY column1, column2 ORDER BY Id) as rn FROM Employee) DELETE FROM CTE WHERE rn > 1;)*
 - **Find the employee not allocated to a project (EmployeeT, ProjectT) using JOIN.**
 - **Brief Answer:**

```
SQL
SELECT E.*
FROM EmployeeT E
LEFT JOIN EmployeeProject EP ON E.employee_id = EP.employee_id
```

WHERE EP.employee_id IS NULL;

- **Retrieve largest to smallest data from SQL.**
 - **Brief Answer:** SELECT * FROM YourTable ORDER BY YourColumn DESC;
- **How to get the number of records from a table (group functions)?**
 - **Brief Answer:** SELECT COUNT(*) FROM YourTable; (for total records).
SELECT column_name, COUNT(*) FROM YourTable GROUP BY column_name; (for grouped counts).
- **Joins: Types of joins (Left, Right, Full, Cross, Self-join). Explain scenario for self-join.**
 - **Brief Answer:**
 - **INNER JOIN:** Returns rows when there is a match in both tables.
 - **LEFT JOIN (or LEFT OUTER JOIN):** Returns all rows from the left table, and matching rows from the right⁴ table. If no match, NULLs from right.
 - **RIGHT JOIN (or RIGHT OUTER JOIN):** Returns all rows from the right table, and matching rows from the left table. If no match, NULLs from left.
 - **FULL JOIN (or FULL OUTER JOIN):** Returns rows when there is a match in one of the tables. Returns all rows when there is a match in either left or right. If no match, NULLs from either side.
 - **CROSS JOIN:** Returns the Cartesian product of the tables (every row from table A combined with every row from table B).
 - **SELF-JOIN:** A table is joined with itself.
 - **Scenario:** Finding employees who report to the same manager, or finding pairs of products that are similar. E.g., SELECT E1.name, E2.name FROM Employees E1 JOIN Employees E2 ON E1.manager_id = E2.manager_id AND E1.id <> E2.id;
- **Keys: Difference between primary key and foreign key. Scenario-based example of primary key, foreign key, composite key.**
 - **Brief Answer:**
 - **Primary Key (PK):** Uniquely identifies each record in a table. Cannot contain NULL values. Only one PK per table.
 - **Foreign Key (FK):** A column (or set of columns) in one table that **refers to the Primary Key** in another table. Establishes a link between tables. Can contain NULL values.
 - **Scenario:**
 - Customers table: customer_id (PK)
 - Orders table: order_id (PK), customer_id (FK referencing Customers.customer_id)
 - **Composite Key:** A primary key composed of **two or more columns** that together uniquely identify a record. E.g., in a StudentCourse table, (student_id, course_id) could be a composite PK to uniquely identify a student's enrollment in a course.
- **Indexing: What is indexing? Scenario-based questions. Disadvantage of indexing.**

Impact of index on query performance. How does SELECT * work internally?

- **Brief Answer:**
 - **Indexing:** A database structure that **improves the speed of data retrieval** operations on a database table. It's like an index in a book.
 - **Scenario:** Indexing customer_id in an Orders table will significantly speed up queries filtering or joining on customer_id.
 - **Disadvantage: Increased storage space, slower write operations** (INSERT, UPDATE, DELETE) because the index also needs to be updated. Over-indexing can degrade performance.
 - **Impact on Query Performance:** Greatly **improves SELECT query performance** for WHERE clauses, JOIN conditions, and ORDER BY clauses by reducing the amount of data the database has to scan.
 - **SELECT * internally:** The database retrieves **all columns** for the matching rows. If an index is used, it first locates the rows efficiently using the index, then fetches all column data for those rows from the actual table data blocks. Using * can be inefficient if you only need a few columns, as it still fetches everything.
- **Normalization: What is it?**
 - **Brief Answer:** A database design technique used to **reduce data redundancy** and improve data integrity. It involves organizing tables and columns to minimize redundant data, typically through a series of "normal forms" (1NF, 2NF, 3NF, BCNF).
- **Triggers: What happens if a trigger fails?**
 - **Brief Answer:** If a database trigger (a stored procedure that automatically runs when a specific event occurs, like INSERT, UPDATE, DELETE) fails, the **DML operation that fired the trigger will typically fail and be rolled back**. The transaction is usually aborted.
- **Query Optimization: How to improve query performance for long-running queries? SQL query optimization steps. Query planner.**
 - **Brief Answer:**
 - **Improvement Steps:**
 - **Indexing** (on frequently queried/joined columns).
 - **Analyze/Explain Plan:** Use EXPLAIN or ANALYZE to understand query execution.
 - **Avoid SELECT *:** Select only necessary columns.
 - **Optimize WHERE clauses:** Avoid functions on indexed columns, use specific conditions.
 - **Proper Joins:** Use appropriate join types.
 - **Limit results:** Use LIMIT or TOP.
 - **Denormalization (judiciously):** For read-heavy, performance-critical scenarios.
 - **Partitioning:** For very large tables.
 - **Query Planner (Optimizer):** The component of the database management

system (DBMS) that **determines the most efficient way to execute a given SQL query**. It considers indexes, table statistics, join orders, and available resources to create an execution plan.

- **Database Locking: How to invoke a lock at the method level? How to handle scenarios where two users try to update at the same time (1st user completes, then 2nd user operates on latest data)? How to avoid multiple writes at the same time from clients?**
 - **Brief Answer:**
 - **Method-level lock:** In Java, use synchronized keyword on a method or block, or use explicit `java.util.concurrent.locks.Lock` (e.g., `ReentrantLock`). In Spring, `@Transactional(isolation = Isolation.SERIALIZABLE)` or optimistic/pessimistic locking via JPA annotations like `@Version` or `@Lock`.
 - **Two users updating:** This is a **concurrency control** problem. Solutions:
 - **Pessimistic Locking:** Lock the row immediately (e.g., `SELECT ... FOR UPDATE`).
 - **Optimistic Locking:** Use a version column (`@Version` in JPA). Each update increments the version. If a concurrent update tries to update an old version, it fails.
 - **Serializable Isolation Level:** Highest isolation, but can cause contention.
 - **Avoid multiple writes:**
 - **Client-side:** Disable submit button, use unique request IDs for idempotency.
 - **Server-side:** Database unique constraints, optimistic/pessimistic locking, idempotent API designs, queueing requests.
- **Database Types: SQL vs. NoSQL (differences, which is better and why). Have you used MongoDB?**
 - **Brief Answer:**
 - **SQL (Relational):**
 - **Structure:** Schema-based, tabular, fixed columns.
 - **Scalability:** Vertical scaling (stronger server), some horizontal with sharding.
 - **ACID:** Strong ACID compliance (Atomicity, Consistency, Isolation, Durability).
 - **Joins:** Strong support for joins.
 - **Use Cases:** Complex transactions, financial systems, applications requiring strong data consistency.
 - **NoSQL (Non-relational):**
 - **Structure:** Schema-less, flexible (document, key-value, column-family, graph).
 - **Scalability:** Horizontal scaling (adding more servers).
 - **ACID:** Typically BASE consistency (Basically Available, Soft State, Eventual Consistency).

- **Joins:** Limited or no native join support; relationships handled at application level.
 - **Use Cases:** Big data, real-time web apps, content management, flexible evolving schemas.
 - **Better:** Depends on the use case. SQL for structured, transactional data. NoSQL for large, unstructured/semi-structured data, high velocity, or high variability.
 - **MongoDB:** Yes/No (state your experience). It's a popular document-oriented NoSQL database.
 - **PreparedStatement vs. CallableStatement: Differences.**
 - **Brief Answer:** Both are used for executing SQL statements, but:
 - **PreparedStatement:** Used for executing **pre-compiled SQL statements**. Good for repetitive queries with varying parameters. **Prevents SQL injection** as parameters are treated as data, not part of the SQL.
 - **CallableStatement:** Used for executing **stored procedures and functions** on the database. Supports input, output, and in-out parameters.
 - **Stored Procedures/Functions: Can we use stored procedures/functions with SELECT queries? If no/yes, why?**
 - **Brief Answer:** Yes.
 - **Stored Procedures:** Can return result sets, which can be consumed by SELECT queries (e.g., EXECUTE procedure_name).
 - **Stored Functions:** Return a single scalar value, which can be directly used within a SELECT statement (e.g., SELECT dbo.myfunction(column) FROM Table;).
 - **Views: What is a view in SQL?**
 - **Brief Answer:** A **virtual table** based on the result-set of a SQL query. It contains rows and columns, just like a real table, but it does not store data itself. It always shows up-to-date data. Used for security (restricting access to certain columns/rows), simplifying complex queries, and data abstraction.
-

Messaging & Asynchronous Communication

- **Messaging Platforms: Have you worked with messaging platforms? If yes, explain.**
 - **Brief Answer:** Yes. (Mention which ones: Kafka, RabbitMQ, ActiveMQ).
 - **Explain:** Messaging platforms enable **asynchronous, decoupled communication** between applications. Producers send messages to topics/queues, and consumers subscribe/listen to these topics/queues to receive and process messages. This helps build resilient, scalable, and loosely coupled distributed systems.
- **Queue vs. Topic: Difference between queue and topic.**
 - **Brief Answer:**
 - **Queue (Point-to-Point):** Messages are sent to a queue and typically

consumed by **only one consumer**. Once a message is consumed, it's removed from the queue. Good for load balancing.

- **Topic (Publish-Subscribe):** Messages are published to a topic and can be consumed by **multiple subscribers/consumers**. Each subscriber receives a copy of the message. Good for broadcasting events.
- **Kafka: Do you know Kafka? Kafka partitions, purpose, role of replicas. Working of Kafka (retention policy). Producer acknowledgements. Different types of acknowledgements in Kafka.**
 - **Brief Answer:** Yes.
 - **Kafka:** A **distributed streaming platform** designed for high-throughput, fault-tolerant real-time data feeds.
 - **Partitions:** Topics are divided into partitions, which are ordered, immutable sequences of records. Partitions enable **parallel processing and scalability** (consumers can read from different partitions concurrently).
 - **Replicas:** Copies of partitions are maintained on different brokers for **fault tolerance and high availability**. If a broker fails, a replica can become the new leader.
 - **Working:** Producers send messages to topics, which are stored in partitions across brokers. Consumers read messages from partitions. Kafka stores messages for a configurable **retention period** (e.g., 7 days) or until a certain size is reached, after which they are deleted.
 - **Producer Acknowledgments (acks):**
 - **acks=0:** Producer doesn't wait for any acknowledgment. Fastest, lowest durability.
 - **acks=1:** Producer waits for the leader partition to write the message.
 - **acks=all (or -1):** Producer waits for all in-sync replicas to confirm the write. Slowest, highest durability.
- **ActiveMQ: If you are using ActiveMQ, what is the load and how are you handling it?**
 - **Brief Answer:** If using ActiveMQ: Describe your messaging volume (e.g., messages/sec). How handled: **Clustering/network of brokers** for high availability and load distribution, using **persistent messages** for durability, **selector-based consumption** for specific message routing, **message prefetch limits** for consumer backpressure.
- **Schedules Tasks: How will you perform scheduling tasks in your project?**
 - **Brief Answer:**
 - **Spring Scheduler (@Scheduled):** For simple, in-application cron-like tasks. Good for single-instance applications or simple cluster-aware solutions.
 - **Quartz Scheduler:** A robust, open-source job scheduling library, suitable for more complex scheduling needs (e.g., job persistence, clustering, dynamic scheduling).
 - **External Schedulers:** If using microservices or need enterprise-grade

scheduling: Kubernetes CronJobs, Airflow, Jenkins, cloud-native schedulers (AWS EventBridge, Azure Scheduler).

Testing & Code Quality

- **Unit Testing: What is unit testing? How to write a good test case?**
 - **Brief Answer:**
 - **Unit Testing:** Testing individual, isolated units of code (methods, classes) to ensure they work as expected.
 - **Good Test Case: F.I.R.S.T. principles:**
 - **Fast:** Runs quickly.
 - **Isolated:** Independent of other tests.
 - **Repeatable:** Produces the same result every time.
 - **Self-validating:** Pass or fail without manual interpretation.
 - **Timely:** Written early, ideally before the code (TDD).
 - Focus on covering edge cases, happy paths, and error paths. Use mocks for external dependencies.
- **Integration Testing: What is integration testing?**
 - **Brief Answer:** Testing the **interactions between different modules or components** of an application (e.g., service layer with database, or two microservices). It ensures that units work correctly when combined.
- **TDD: What is TDD (Test-Driven Development)?**
 - **Brief Answer:** A software development process where **tests are written before the code** they are meant to test. The cycle is: **Red** (write failing test), **Green** (write minimal code to pass test), **Refactor** (improve code).
- **JUnit: How to write a good test case? @Ignore and @Disable annotations. How to define mock and injectMock?**
 - **Brief Answer:**
 - **Good Test Case:** See "Unit Testing" above. In JUnit, use @Test, @BeforeEach, @AfterEach, assertions (assertEquals, assertTrue).
 - **@Ignore (JUnit 4):** Skips the annotated test method/class.
 - **@Disabled (JUnit 5):** Same as @Ignore for JUnit 5.
 - **Mockito (@Mock, @InjectMocks):**
 - **@Mock:** Creates a **mock object** (a dummy object that mimics a real one), allowing you to control its behavior and verify interactions. when(...).thenReturn(...).
 - **@InjectMocks:** Injects mocked dependencies (@Mock objects) into the object under test.
- **Mockito: Difference between @Mock and @Spy.**
 - **Brief Answer:**
 - **@Mock:** Creates a **pure mock** where all methods (unless stubbed) do nothing and return default values. You control all behavior.

- **@Spy:** Creates a **partial mock (spy)** around a real object. By default, it calls the real methods, but you can selectively stub certain methods. Use when you want to test a real object but mock specific interactions.
 - **Code Quality: How to maintain code quality?**
 - **Brief Answer:**
 - **Coding Standards:** Follow style guides (e.g., Google Java Style Guide).
 - **Code Reviews:** Peer reviews to catch issues and share knowledge.
 - **Static Analysis Tools:** SonarQube, Checkstyle, PMD to detect code smells, bugs, vulnerabilities.
 - **Automated Testing:** High test coverage (unit, integration).
 - **Refactoring:** Continuously improve code structure without changing external behavior.
 - **Documentation:** Clear comments, Javadocs.
 - **Clear Naming Conventions.**
 - **Database Testing: How to write unit test cases for a database?**
 - **Brief Answer:** For unit tests, you generally **mock the database layer** (e.g., JdbcTemplate, EntityManager, Repository) using Mockito to isolate the business logic. For **integration tests**, you'd use an in-memory database (e.g., H2) or a test container (e.g., Testcontainers library) to spin up a real database instance for testing.
-

DevOps & Deployment

- **CI/CD: How CI/CD works? What CI/CD process do you follow? How do you handle CI/CD pipeline in your project?**
 - **Brief Answer:**
 - **CI (Continuous Integration):** Developers frequently merge code into a central repository, where automated builds and tests are run. Goal is to catch integration issues early.
 - **CD (Continuous Delivery/Deployment):**
 - **Continuous Delivery:** Code is always in a deployable state, ready for manual release to production.
 - **Continuous Deployment:** Code is automatically deployed to production after passing all tests.
 - **Process:** Describe your process (e.g., Developers push code to Git -> Jenkins/GitLab CI/GitHub Actions triggered -> Build, Unit Tests, Integration Tests -> Docker Image Build -> Image pushed to Registry -> Deployment to lower environments -> Manual/Automated approval -> Deployment to Prod).
- **Deployment: How do you deploy your applications? Blue/Green deployment.**
 - **Brief Answer:**
 - **Deployment:** Describe your method (e.g., **Containerization (Docker) and Orchestration (Kubernetes)**); direct VM deployment; cloud services like

AWS Elastic Beanstalk/ECS).

- **Blue/Green Deployment:** A strategy for zero-downtime deployment. You maintain two identical production environments ("Blue" for current version, "Green" for new version). New traffic is routed to "Green" after it's tested. If issues arise, traffic can quickly be rolled back to "Blue". "Blue" can then be retired or become the new "Green" for the next release.
- **Jenkins: Have you used Jenkins?**
 - **Brief Answer:** Yes. Jenkins is an **open-source automation server** used to automate parts of the software development process, including building, testing, and deploying.⁵ Explain how you used it (e.g., setting up pipelines, integrating with SCM, running tests, deploying artifacts).
- **Docker: Docker file and Docker commands. How to create a Docker file? How to mount a local disk to a container? Command to find a container in Docker. Structure of a Docker file.**
 - **Brief Answer:**
 - **Docker:** A platform that uses **containerization** to package an application and its dependencies into a lightweight, portable unit (Docker image), which can then run consistently across any environment.
 - **Docker File:** A text file containing instructions for **building a Docker image**.
 - **Basic Structure:** FROM (base image), WORKDIR (working directory), COPY (copy files), RUN (execute commands), EXPOSE (port), CMD (default command to run).
 - **Mount Local Disk:** Use the -v or --volume flag: docker run -v /local/path:/container/path image_name.
 - **Find Container:** docker ps (running), docker ps -a (all).
- **Kubernetes: Explain Kubernetes. How does it work? How did you use it in your project?**
 - **Brief Answer:**
 - **Kubernetes (K8s):** An **open-source container orchestration platform** that automates the deployment, scaling, and management of containerized applications.
 - **How it works:** It uses a **cluster** of nodes (master and worker nodes). You define desired states (e.g., 3 replicas of an Nginx web server) using YAML manifests. Kubernetes continuously works to maintain that state by scheduling containers, managing networking, storage, and handling failures.
 - **Usage:** Used it to deploy, manage, and scale microservices. (Mention specific K8s objects like Deployments, Services, Ingress, Pods, ConfigMaps, Secrets, PersistentVolumes).
- **OpenShift: How does it work? How did you use it in your project?**
 - **Brief Answer:** OpenShift is a **Red Hat product** that provides a **Kubernetes distribution with added enterprise-grade features** and developer tooling. It builds on Kubernetes by adding features like built-in CI/CD, source-to-image (S2I)

capabilities, developer consoles, and enterprise security. Used it as the platform for deploying, managing, and scaling our containerized applications, benefiting from its integrated tools and security.

- **Production Debugging: How to debug a production environment? Which tools have you used?**
 - **Brief Answer:**
 - **Logging:** Centralized logging (ELK stack - Elasticsearch, Logstash, Kibana; Splunk, Grafana Loki) is crucial.
 - **Monitoring & Alerting:** Prometheus, Grafana, Datadog, New Relic to track metrics (CPU, memory, latency, error rates) and trigger alerts.
 - **Distributed Tracing:** Jaeger, Zipkin, Spring Cloud Sleuth to trace requests across multiple microservices.
 - **Profiling Tools:** JVisualVM, JProfiler (if allowed in prod, usually for specific issues), jstack for thread dumps.
 - **Heap Dumps:** jmap for memory analysis.
 - **Remote Debugging:** Enabled cautiously and temporarily.
- **Maven: Maven scopes (default scope). How to resolve dependency conflicts in Maven?**
 - **Brief Answer:**
 - **Maven Scopes:** Control the classpath for different build phases (compile, test, runtime, provided, system, import).
 - **compile (default):** Available in all classpaths.
 - **test:** Only for compilation/execution of tests.
 - **runtime:** Required at runtime, not compile time.
 - **provided:** Provided by JDK or web server (e.g., servlet-api).
 - **system:** Specific path on local system.
 - **import:** Used in dependency management to import a BOM.
 - **Resolve Conflicts:**
 - **Dependency Exclusion:** Exclude a transitive dependency that causes conflict.
 - **Dependency Mediation:** Maven's "nearest definition" rule (the dependency closest to your project in the dependency tree wins). Override by explicitly declaring the desired version.
 - **mvn dependency:tree:** To visualize and analyze the dependency tree.

System Design & Architecture

- **Large File Upload: What is your approach to upload a large file into DB? How to upload a file into the DB if it has huge data (JSON/CSV, full flow and approach)?**
 - **Brief Answer:**
 - **Avoid storing large files directly in DB (BLOB/CLOB)** unless absolutely

necessary (e.g., very small files, specific audit requirements). It can bloat DB, slow backups, and impact performance.

- **Preferred Approach:**
 1. **Store files in external object storage:** (e.g., AWS S3, Azure Blob Storage, MinIO).
 2. **Database stores metadata:** Store file name, path/URL, size, content type, upload date, owner, and a unique identifier in the DB.
 3. **Chunking/Multipart Upload:** For extremely large files, use chunking (breaking into smaller pieces) and multipart upload APIs provided by storage services.
 4. **Asynchronous Processing:** Upload to storage, then use a message queue (Kafka/RabbitMQ) to notify a processing service to handle metadata update or further processing (e.g., parsing CSV, validation).
- **System Communication: How will you communicate between two different systems? How does a POST method call from one system to another work?**
 - **Brief Answer:**
 - **Communication:**
 - **Synchronous:** REST APIs (HTTP/JSON), gRPC.
 - **Asynchronous:** Message Queues (Kafka, RabbitMQ), Event Streaming.
 - **File Transfer:** SFTP, shared storage.
 - **Database:** Direct database access (less common for different systems, more for shared data).
 - **POST Method Call:** System A (client) constructs an HTTP POST request (with body containing data, headers). This request is sent over the network to System B's (server) specific endpoint. System B's server processes the request, performs operations (e.g., creates a resource), and sends back an HTTP response (status code like 201 Created, 200 OK, and potentially a response body).
- **Multiple Documents Storage: How will you store multiple documents? How will you store multiple loan documents?**
 - **Brief Answer:** Same as large file upload: **Store documents in an object storage solution (S3, Azure Blob, Google Cloud Storage)**. The database stores only the **metadata** (document ID, type, file path/URL in S3, version, etc.) and relationships to entities (e.g., loan_id). This scales better, is more cost-effective, and performance-friendly than storing blobs in a relational DB.
- **Designing 3 Applications (20+ fields): How would you design them? What technology stack would you use? How would you provide an estimate?**
 - **Brief Answer:**
 - **Design:**
 - **Microservices:** If independent domains/teams, scale needed.
 - **Modular Monolith:** If tight coupling or initial stage.
 - **Domain-Driven Design (DDD):** Identify bounded contexts for each

application/module.

- **Shared Components:** Extract common fields/logic into shared libraries (e.g., DTOs, utility classes, common security module).
- **API Design:** Clearly defined RESTful APIs for inter-application communication.
- **Database Strategy:** Separate databases per application or logical separation within a shared DB (if monolithic).
- **Technology Stack:**
 - **Backend:** Java, Spring Boot (fast development, robust ecosystem).
 - **Database:** PostgreSQL/MySQL (relational, transactional) or potentially NoSQL (MongoDB/Cassandra) if specific needs like high write volume, flexible schema.
 - **Frontend:** React/Angular (modern SPAs).
 - **Messaging:** Kafka/RabbitMQ (for async communication).
 - **Deployment:** Docker, Kubernetes (for containerization & orchestration).
 - **CI/CD:** Jenkins/GitLab CI.
 - **Monitoring:** Prometheus/Grafana, ELK stack.
- **Estimate:**
 - **Breakdown:** Break down into smaller, manageable tasks (features, modules, stories).
 - **Techniques:**
 - **Bottom-Up:** Estimate each task, sum up.
 - **Analogous/Expert Judgment:** Compare to similar past projects.
 - **Parametric:** Use metrics from past projects.
 - **Three-point Estimation:** (Optimistic + 4*Most Likely + Pessimistic) / 6.
 - **Considerations:** Requirements clarity, team experience, third-party dependencies, known risks, buffer/contingency.
 - **Agile:** Incremental estimates per sprint.
- **API Performance: How do you measure the performance of your API? What do you do if the API handles large data?**
 - **Brief Answer:**
 - **Measure:**
 - **Metrics:** Response time (latency), throughput (requests/sec), error rate, resource utilization (CPU, memory, network I/O).
 - **Tools:** Load testing tools (JMeter, Gatling), APM tools (New Relic, Datadog), Prometheus/Grafana, Spring Actuator.
 - **Handling Large Data:**
 - **Pagination:** Limit number of records returned per request.
 - **Filtering & Sorting:** Allow clients to filter/sort results.
 - **Compression:** Enable GZIP/Brotli compression for responses.

- **Streaming:** For very large responses, stream data instead of loading all into memory.
 - **Caching:** Cache frequently accessed large data.
 - **Asynchronous Processing:** If data needs transformation, process asynchronously.
 - **Optimized Queries:** Ensure database queries are highly optimized.
- **Security: How will you access entities using security? How is security maintained in your application?**
 - **Brief Answer:** (Covered in Spring Security section).
- **Intranet/Internet Application: Is your project an intranet or internet application?**
 - **Brief Answer:** State whether it's for internal use (intranet) or public access (internet). This impacts security, networking, and deployment considerations significantly.
- **Read Data When Available: Requirement to read data from some place when it will be available. Your approach.**
 - **Brief Answer:** This suggests an **event-driven or polling approach**.
 - **Event-Driven:** If the "place" can publish events, use a **message queue/streaming platform (Kafka)**. A consumer service subscribes to these events and processes the data immediately when available.
 - **Polling:** If events aren't available, periodically poll the source (e.g., check an S3 bucket, FTP server, or external API endpoint) using a **scheduler (Spring @Scheduled or Quartz)**. Be mindful of polling frequency and potential API rate limits.
- **JVM Out of Memory: How to handle JVM out of memory situations? What is your approach? Scenario when a huge CSV file is larger than JVM memory, how to find the largest number?**
 - **Brief Answer:**
 - **Handle OOM:**
 - **Monitor:** Set up alerts for memory usage (e.g., via Prometheus, Grafana).
 - **Troubleshoot:** Analyze **heap dumps** (using jmap, Eclipse MAT, VisualVM) to identify memory leaks, large objects, or inefficient data structures.
 - **Increase Heap Size:** Adjust JVM **-Xmx** (max heap) and **-Xms** (initial heap) parameters.
 - **Optimize Code:** Release unused objects, use efficient collections, avoid creating excessive temporary objects, lazy loading.
 - **Garbage Collector Tuning:** Experiment with different GC algorithms.
 - **Huge CSV file (larger than JVM memory) - find largest number:**
 - **External Sorting:** Read chunks of the file, sort each chunk, write back to temporary files. Then merge the sorted chunks.
 - **Stream Processing:** Read the file line by line using a **BufferedReader or Scanner**. Parse each number one by one,

keeping track of only the largest number found so far. This avoids loading the entire file into memory.

- **Specialized Libraries:** Use libraries designed for large data processing (e.g., Apache Spark, Hadoop).
- **Concurrency Problem: Solution for concurrency problems. How to avoid multiple writes at the same time from clients.**
 - **Brief Answer:**
 - **Solutions:**
 - **Synchronization:** synchronized keyword, `java.util.concurrent.locks.Lock` (`ReentrantLock`, `ReadWriteLock`).
 - **Atomic Operations:** `java.util.concurrent.atomic` package (e.g., `AtomicInteger`).
 - **Concurrent Collections:** `ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue`.
 - **Thread-Safe Design:** Immutable objects, careful state management.
 - **Database Transactions & Locking:** Pessimistic/Optimistic locking.
 - **Multiple Writes from Clients:** (Covered in Database Locking section).
- **Synchronization: How are you using ThreadPool and synchronization in your project?**
 - **Brief Answer:**
 - **ThreadPool:** Used `ExecutorService` (e.g., `FixedThreadPool`, `CachedThreadPool`) to manage and reuse threads for concurrent task execution, avoiding overhead of creating new threads for each task. Good for managing resources and ensuring a controlled number of concurrent operations.
 - **Synchronization:** Applied synchronized blocks/methods or `ReentrantLock` when accessing shared mutable resources (e.g., a counter, a shared cache, database connections) to ensure thread safety and prevent race conditions.
- **Memory Leakage: What is it? How to identify it? Solutions for memory leakage.**
 - **Brief Answer:**
 - **Memory Leakage:** Occurs when an application unintentionally holds onto objects that are no longer needed, preventing the garbage collector from reclaiming their memory. Over time, this leads to increased memory consumption and eventually `OutOfMemoryError`.
 - **Identify:**
 - **Monitoring:** Observing continuous increase in heap memory usage over time (even after GC cycles).
 - **Profiling Tools:** Use JVM profilers (`JVisualVM`, `JProfiler`, `YourKit`) to analyze heap dumps, identify unreachable objects, and track object allocations.
 - **Heap Dumps:** Analyze objects, references, and GC roots.
 - **Solutions:**
 - **Remove strong references:** Ensure objects that are no longer

- needed are dereferenced.
 - **Close resources:** Properly close database connections, streams, file handles.
 - **Event listeners:** Remove listeners when objects are no longer required.
 - **Static collections:** Be careful with static collections, as they can hold references for the application's lifetime.
 - **Cache management:** Implement proper cache eviction policies.
 - **Split Requirements: How to split a requirement into multiple microservices? What's the deciding factor?**
 - **Brief Answer:**
 - **Split Requirements:** Primarily based on **business capabilities or bounded contexts** (Domain-Driven Design). Each service should ideally own its data and be independently deployable. Look for natural functional boundaries where components communicate minimally.
 - **Deciding Factors:**
 - **Business Domain:** Core business capabilities (e.g., Order, Customer, Product).
 - **Autonomy:** Services should be owned by small, independent teams.
 - **Scalability Needs:** Components with high traffic/scaling needs can be separated.
 - **Technology Heterogeneity:** If different tech stacks are desired for different parts.
 - **Team Size/Structure:** Aligned with Conway's Law.
-

Threads & Concurrency

- **Threads: What is a thread? Thread lifecycle. Thread states (Blocked, Waiting, Timed Waiting). How to execute multiple threads?**
 - **Brief Answer:**
 - **Thread:** A **lightweight unit of execution** within a process. A single process can have multiple threads running concurrently.
 - **Lifecycle:** **New, Runnable, Running, Blocked, Waiting, Timed Waiting, Terminated.**
 - **States:**
 - **NEW:** Thread has been created but not yet started.
 - **RUNNABLE:** Thread is executing in the JVM or ready to be executed (waiting for processor).
 - **BLOCKED:** Thread is waiting for a monitor lock to enter a synchronized block/method.
 - **WAITING:** Thread is waiting indefinitely for another thread to perform a particular action (e.g., `Object.wait()`, `Thread.join()`).

- **TIMED_WAITING:** Thread is waiting for another thread to perform an action for a specified waiting time (e.g., Thread.sleep(long), Object.wait(long)).
 - **TERMINATED:** Thread has completed its execution.
 - **Execute Multiple Threads:**
 - Extend Thread class and override run().
 - Implement Runnable interface and pass to Thread constructor.
 - Use ExecutorService (preferred way) with Runnable or Callable for managing thread pools.
- **ExecutorService: Types of ExecutorService (FixedThreadPool, CachedThreadPool). What is a priority queue and a blocking queue? How does it differentiate I/O, System, and Other operations? Explain scenario for choosing thread pool. Attributes of thread pool.**
 - **Brief Answer:**
 - **ExecutorService:** A high-level API in java.util.concurrent for managing and controlling threads. It provides a way to decouple task submission from thread execution.
 - **Types:**
 - **FixedThreadPool:** Creates a fixed number of threads. Tasks are put into a queue if all threads are busy. Good for CPU-bound tasks.
 - **CachedThreadPool:** Creates new threads as needed, but reuses existing idle threads. Good for I/O-bound tasks with many short-lived operations.
 - **SingleThreadExecutor:** A single worker thread.
 - **ScheduledThreadPoolExecutor:** For scheduling tasks after a delay or periodically.
 - **Priority Queue:** Elements are retrieved based on their priority.
 - **Blocking Queue:** A queue that blocks when adding or removing elements if the queue is full/empty. Essential for producer-consumer patterns.
 - **Differentiation:** Thread pools don't automatically differentiate I/O, System, etc. You choose the **appropriate thread pool type based on the nature of the tasks** (e.g., CachedThreadPool for many I/O-bound tasks, FixedThreadPool for CPU-bound).
 - **Scenario for choosing:** For a web server handling many concurrent requests (I/O-bound operations like database calls), a CachedThreadPool might be efficient. For background processing of a fixed number of CPU-intensive calculations, a FixedThreadPool matching core count would be better.
 - **Attributes of Thread Pool:** Core pool size, maximum pool size, keep-alive time, work queue (blocking queue), thread factory, rejected execution handler.
- **ForkJoinPool: What is its functionality?**
 - **Brief Answer:** A specialized ExecutorService designed for **divide-and-conquer**

algorithms (e.g., parallel processing of arrays). It's optimized for tasks that can be broken down into smaller subtasks, and uses a **work-stealing algorithm** to efficiently distribute tasks among threads.

- **join() method: Explain join() in threads.**

- **Brief Answer:** The join() method on a thread instance causes the **current thread to wait until the thread it's joining with finishes its execution**. thread.join() makes the current thread wait until thread dies.

- **Deadlock: What is it? How to identify a deadlock if you only have server access? Key identifiers in a thread dump?**

- **Brief Answer:**
 - **Deadlock:** A situation where **two or more threads are blocked indefinitely**, each waiting for the other to release a resource that it needs.⁶
 - **Identify (server access):** Take **thread dumps** (jstack <pid>). Look for threads in BLOCKED or WAITING state, specifically looking for "waiting for monitor entry" or "waiting on" messages, and then look for "locked <address>" in other threads that own those monitors.
 - **Key Identifiers in Thread Dump:**
 - "Deadlock detected:" (sometimes explicitly by JVM).
 - "waiting for monitor entry [0x...]".
 - "locked <0x...>".
 - Circular dependencies of locks.

- **Virtual Threading: What is it?**

- **Brief Answer:** (Introduced in Java 19 as preview, targeted for full release soon - Project Loom). Virtual threads are **lightweight threads** managed by the JVM, not directly mapped to OS threads. They allow for **massive concurrency** (millions of virtual threads) without consuming significant OS resources. They are designed to simplify writing high-throughput, I/O-bound concurrent applications by allowing a "thread-per-request" style without the overhead of traditional platform threads.

- **Lock Interface: What is it? How to invoke a lock at the method level?**

- **Brief Answer:**
 - **Lock Interface:** A more flexible and powerful alternative to the synchronized keyword for thread synchronization. It provides methods like lock(), unlock(), tryLock(), and newCondition().

- **Invoke at Method Level:**

Java

```
import java.util.concurrent.locks.Lock;
```

```
import java.util.concurrent.locks.ReentrantLock;
```

```
public class MyClass {
```

```
    private final Lock lock = new ReentrantLock();
```

```
    public void mySynchronizedMethod() {
```

```
        lock.lock(); // Acquire the lock
```

```

    try {
        // Critical section: code that needs to be thread-safe
        // ...
    } finally {
        lock.unlock(); // Release the lock in a finally block
    }
}
}

```

Other Technical Questions

- **HTTP Methods: Different types of HTTP methods and their use. Difference between GET and PUT methods.**
 - **Brief Answer:**
 - **Methods:**
 - GET: Retrieve data. Idempotent & Safe.
 - POST: Submit data to be processed (e.g., create a resource). Not idempotent.
 - PUT: Update a resource entirely or create if it doesn't exist. Idempotent.
 - DELETE: Delete a resource. Idempotent.
 - PATCH: Apply partial modifications to a resource. Not necessarily idempotent.
 - HEAD: Same as GET, but without the response body.
 - OPTIONS: Describes communication options for the target resource.
 - **GET vs. PUT:**
 - **GET:** Retrieves data. Has no side effects on the server state. Used for reading.
 - **PUT:** Updates an existing resource completely or creates a new one if it doesn't exist. It's **idempotent**, meaning calling it multiple times with the same input has the same effect as calling it once.
- **HTTP Status Codes: Explain HTTP status codes and their meaning.**
 - **Brief Answer:** Three-digit numbers returned by a server to indicate the status of a request.
 - **1xx (Informational):** Request received, continuing process.
 - **2xx (Success):** Action successfully received, understood, and accepted.
 - 200 OK: Standard success.
 - 201 Created: Resource successfully created (typically for POST).
 - 204 No Content: Successful, but no content to return.
 - **3xx (Redirection):** Further action needs to be taken.
 - 301 Moved Permanently.

- **4xx (Client Error):** Request contains bad syntax or cannot be fulfilled.
 - 400 Bad Request: General client error.
 - 401 Unauthorized: Client needs to authenticate.
 - 403 Forbidden: Client doesn't have access.
 - 404 Not Found: Resource not found.
 - 409 Conflict: Conflict with current state of resource.
 - **5xx (Server Error):** Server failed to fulfill an apparently valid request.
 - 500 Internal Server Error: General server error.
 - 503 Service Unavailable: Server temporarily unable to handle.
 - **Idempotent: What is it?**
 - **Brief Answer:** An operation is idempotent if executing it multiple times with the same parameters has the **same effect** as executing it once. (e.g., GET, PUT, DELETE). Non-idempotent operations (POST) can have different effects each time.
 - **Versioning: Have you used version control (e.g., SVN, Git/Bitbucket)?**
 - **Brief Answer:** Yes. (State which one(s) and how you use them - branching, merging, pull requests, commit best practices).
 - **IDE: Which IDE do you use for React/Java?**
 - **Brief Answer:** For Java: IntelliJ IDEA (Community/Ultimate), Eclipse, VS Code with Java extensions. For React: VS Code. (Mention why you prefer it: features, extensions, performance).
 - **Optional.of vs. Optional.ofNullable: Difference.**
 - **Brief Answer:** (Covered in Optional class section).
 - **Recursion Function in Lambda: Example.**
 - **Brief Answer:** While direct recursion in a simple lambda is tricky due to final variable capture, you can achieve it using a wrapper or specific patterns.
- Java
- ```
// Example using a wrapper/interface for recursion in lambda
interface FactorialFunction {
 long factorial(long n);
}
```
- ```
FactorialFunction factorial = (n) -> {
    if (n == 0) return 1;
    // This is the tricky part, needs an external reference or special techniques for
    // direct lambda recursion
    // A simple example for a stream operation, not a recursive lambda itself:
    // list.stream().filter(x -> x < someValue).map(...).collect(...)
    // A more complex example for actual recursion often involves an inner class or
    // a fixed-point combinator.
    // For interview, focus on explaining the concept of recursive lambda if possible.
    // Simpler: Just say Lambdas can call recursive methods.
    return n * factorial(n - 1); // This would require 'factorial' to be effectively final
}
```



```
};
```

// A common pattern for self-referential lambdas is using a functional interface as a holder

```
// Function<Long, Long> factorial = null;
```

// factorial = (n) -> (n == 0) ? 1 : n * factorial.apply(n - 1); // This is not directly allowed without workarounds

Better answer: "While direct self-referential recursion within a simple lambda is limited by Java's final capture rules, lambdas can effectively *call* recursive methods. For true recursive lambda expressions (like (n) -> n * factorial.apply(n-1)), you often need more advanced patterns like using a Function instance as a holder or a Y-combinator type pattern, which are more academic than practical for typical day-to-day use."

- **Java 8 to Java 17 Migration: What changes do you need to make?**
 - **Brief Answer:**
 - **Language Features:** No major breaking changes (mostly additions like var, records, pattern matching, sealed classes). May use new features if desired.
 - **Module System (JPMS):** If using modules, may need to define module-info.java files.
 - **Library Updates:** Update Spring Boot to compatible versions (e.g., Spring Boot 2.5+ or 3.x for Java 17). Update other third-party libraries.
 - **Removed Features:** Awareness of removed APIs (e.g., Nashorn JavaScript engine, sun.misc.Unsafe internal APIs usage - check library dependencies).
 - **GC Improvements:** Benefit from newer GCs (G1 default, ZGC/Shenandoah).
 - **JVM Arguments:** Some JVM flags may have changed or been removed.
- **Hadoop, Big Data, Spark, Kafka and others: Expecting questions on these (if relevant to your background).**
 - **Brief Answer:** (State your experience). If you have no direct experience, explain foundational concepts. E.g., "While I haven't worked directly with Hadoop, I understand its role in distributed storage (HDFS) and processing (MapReduce) for big data. I have worked with Kafka, which is a key component in modern data streaming architectures."
- **Angular Code Comfort: How comfortable are you writing Angular code?**
 - **Brief Answer:** Be honest. "Comfortable with basic CRUD operations, component creation, routing, form handling, and integrating with REST APIs." Or, "Have foundational knowledge but primarily a backend developer."

Behavioral & Project-Specific

- **Tell me about yourself.**
 - **Brief Answer:** Start with your **current role and experience**, highlight **key skills** relevant to the job, mention a **recent achievement or project**, and briefly state

your **career aspirations** or what you're looking for. Keep it concise, positive, and focused on professional aspects.

- **Explain your recent project (technically).**
 - **Brief Answer: STAR method (Situation, Task, Action, Result).**
 - **Situation:** Briefly describe the project's domain/purpose.
 - **Task:** Your specific role and responsibilities.
 - **Action:** Technical challenges faced and solutions implemented (mention specific technologies, architecture, patterns used).
 - **Result:** Quantifiable achievements (e.g., improved performance by X%, reduced errors by Y%, supported Z users).
 - *Be ready to deep-dive into any technology mentioned.*
- **What agile process do you follow?**
 - **Brief Answer:** (e.g., **Scrum**). Explain key ceremonies: **Sprint Planning, Daily Stand-ups, Sprint Review, Sprint Retrospective**. Mention roles (Product Owner, Scrum Master, Development Team) and artifacts (Product Backlog, Sprint Backlog, Increment).
- **How do you measure performance of your API?**
 - **Brief Answer:** (Covered in System Design -> API Performance).
- **How do you monitor and handle memory issues in your project?**
 - **Brief Answer:** (Covered in System Design -> JVM Out of Memory).
- **How do you handle deployment in production if any memory issue happens?**
 - **Brief Answer:** If a memory issue occurs during deployment:
 1. **Immediate Action:** Rollback to the previous stable version (if using Blue/Green or similar).
 2. **Investigation:** Collect thread dumps, heap dumps, analyze logs, and metrics.
 3. **Root Cause Analysis:** Identify if it's a code-related leak, misconfiguration (e.g., insufficient memory allocation), or unexpected load.
 4. **Resolution:** Fix code, adjust JVM memory, or scale up resources.
 5. **Prevention:** Implement pre-deployment memory/load tests, better monitoring, and review code for potential leaks.
- **How much do you rate yourself in Java?**
 - **Brief Answer:** Be honest but confident. Avoid 10/10. "I'd rate myself an 8/10. I have a strong grasp of core Java, Java 8 features, and the Spring ecosystem. I'm always learning and staying updated with new features and best practices."

Coding Challenges

- **Uppercase to Lowercase & Vice-versa:** Convert uppercase characters in a String to lowercase and vice-versa (Java 8).

Java

```
public String toggleCase(String str) {
```

```

return str.chars()
    .mapToObj(c -> {
        if (Character.isUpperCase(c)) {
            return Character.toLowerCase(c);
        } else if (Character.isLowerCase(c)) {
            return Character.toUpperCase(c);
        }
        return (char) c;
    })
    .collect(StringBuilder::new, StringBuilder::appendCodePoint,
StringBuilder::append)
    .toString();
}

```

- **Group Employees by Salary:** Group a list of employees based on salary (Java 8).

Java

```
import java.util.List;
```

```
import java.util.Map;
```

```
import java.util.stream.Collectors;
```

```
class Employee {
```

```
    String name;
```

```
    double salary;
```

```
    // Constructor, getters, setters
```

```
    public Employee(String name, double salary) { this.name = name; this.salary = salary;
```

```
}
```

```
    public double getSalary() { return salary; }
```

```
    public String getName() { return name; }
```

```
    @Override public String toString() { return name + "(" + salary + ")"; }
```

```
}
```

```
public Map<Double, List<Employee>> groupEmployeesBySalary(List<Employee>
employees) {
```

```
    return employees.stream()
```

```
        .collect(Collectors.groupingBy(Employee::getSalary));
```

```
}
```

- **Return 0 for 1, 1 for 0: Return 0 if input is 1, and 1 if input is 0, *without* using if/else, ternary, switch, or other inputs.**

Java

```
public int flip(int input) {
```

```
    return 1 - input; // Works for 0 and 1
```

```
}
```

// Other options: input ^ 1 (bitwise XOR with 1), or Math.abs(input - 1)

- **Find Second Largest Salary: Get the employee with the second largest salary of a specific department using streams (in case multiple employees have the same salary).**

Java

```
import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;
```

// Assume Employee class as above, with a String department field

```
class Employee {
    String name;
    double salary;
    String department;
    public Employee(String name, double salary, String department) { /* ... */ }
    public double getSalary() { return salary; }
    public String getDepartment() { return department; }
    // ... toString, equals, hashCode
}
```

```
public List<Employee> getEmployeesWithSecondLargestSalary(List<Employee>
employees, String department) {
```

```
    Optional<Double> secondLargestSalary = employees.stream()
        .filter(e -> e.getDepartment().equals(department))
        .map(Employee::getSalary)
        .distinct() // Handle multiple employees with same salary
        .sorted(Comparator.reverseOrder())
        .skip(1) // Skip the largest salary
        .findFirst(); // Get the second largest salary
```

```
    if (secondLargestSalary.isPresent()) {
        double targetSalary = secondLargestSalary.get();
        return employees.stream()
            .filter(e -> e.getDepartment().equals(department) && e.getSalary() ==
targetSalary)
            .collect(Collectors.toList());
    }
    return List.of(); // Return empty list if no such salary found
}
```

- **Notify Customer (3 days before creation date): Using Java 8, write code to notify**

a customer if the creation date is 3 days before the current date.

Java

```
import java.time.LocalDate;
```

```
import java.util.List;
```

```
import java.util.stream.Collectors;
```

```
class Customer {
```

```
    String name;
```

```
    LocalDate creationDate;
```

```
    // Constructor, getters
```

```
    public Customer(String name, LocalDate creationDate) { this.name = name;
```

```
    this.creationDate = creationDate; }
```

```
    public LocalDate getCreationDate() { return creationDate; }
```

```
    public String getName() { return name; }
```

```
}
```

```
public List<Customer> getCustomersToNotify(List<Customer> customers) {
```

```
    LocalDate threeDaysAgo = LocalDate.now().minusDays(3);
```

```
    return customers.stream()
```

```
        .filter(customer -> customer.getCreationDate().isEqual(threeDaysAgo))
```

```
        .collect(Collectors.toList());
```

```
}
```

- **Reverse Linked List:** Reverse a linked list without using recursion.

Java

```
// Assuming a simple Node class for a singly linked list
```

```
class Node {
```

```
    int data;
```

```
    Node next;
```

```
    Node(int data) { this.data = data; this.next = null; }
```

```
}
```

```
public Node reverseLinkedList(Node head) {
```

```
    Node prev = null;
```

```
    Node current = head;
```

```
    while (current != null) {
```

```
        Node nextTemp = current.next; // Store next node
```

```
        current.next = prev;          // Reverse current node's pointer
```

```
        prev = current;               // Move prev to current node
```

```
        current = nextTemp;           // Move current to next node
```

```
    }
```

```
    return prev; // prev will be the new head
```

```
}
```

- **Missing Number:** Find the missing number (e.g., between 1 and 100).

Java

```
public int findMissingNumber(int[] numbers, int n) {
    long expectedSum = (long) n * (n + 1) / 2;
    long actualSum = 0;
    for (int num : numbers) {
        actualSum += num;
    }
    return (int) (expectedSum - actualSum);
}
```

- **Remove Duplicates in Array (in order) / Sort Array Small to Large:** Standard Set for duplicates, Arrays.sort() for sorting. Custom implementations would be too long for brief answer.

- **List Operations (Stream API):**

- **List of {1,2,3,4,5}, return 3,4 as a list without using sorting, comparison and filter but need to use stream.**

Java

```
import java.util.List;
import java.util.stream.Collectors;
```

// This is tricky without 'filter' or 'comparison'. It implies 'map' or 'reduce' based logic.

// If the intent is to get elements at specific indices, that's different.

// Assuming a mapping based on values that happen to be 3 and 4, this is highly specific:

```
public List<Integer> getSpecificNumbers(List<Integer> numbers) {
    return numbers.stream()
        .filter(n -> n == 3 || n == 4) // This is the only way to select without
        "comparison"
        .collect(Collectors.toList());
}
```

// If "no filter" means literally no .filter() method, then it becomes very contrived or implies a

// fixed size input where you know indices. For example, if input is always 1,2,3,4,5:

```
public List<Integer> getSpecificNumbersNoFilter(List<Integer> numbers) {
    return List.of(numbers.get(2), numbers.get(3)); // Assumes list is 0-indexed and
    contains 1,2,3,4,5
}
```

// This question phrasing is unusual for "no filter, no comparison" with arbitrary lists.

- **Group 0s and numbers separately and return a single list using streams without using filter and sorting.**

Java

// This is complex without filter/sorting. A collector might work, but it's still
// doing a form of implicit "filtering" by putting 0s first.

// Typically, you'd use a partitioningBy collector:

// Map<Boolean, List<Integer>> partitioned =
numbers.stream().collect(Collectors.partitioningBy(n -> n == 0));

// List<Integer> result = new ArrayList<>(partitioned.get(true));

// result.addAll(partitioned.get(false));

// If strictly no filter/sorting: Requires a custom collector or a non-stream
approach

```
public List<Integer> groupZerosAndNumbers(List<Integer> numbers) {
    List<Integer> zeros = new java.util.ArrayList<>();
    List<Integer> nonZeros = new java.util.ArrayList<>();
    numbers.forEach(n -> {
        if (n == 0) zeros.add(n);
        else nonZeros.add(n);
    });
    zeros.addAll(nonZeros);
    return zeros;
}
```

- **Find the longest string in a list using streams.**

Java

```
public Optional<String> findLongestString(List<String> strings) {
    return strings.stream()
        .max(Comparator.comparingInt(String::length));
}
```

- **Find duplicate elements from a list (using Java 8 stream).**

Java

import java.util.HashSet;

import java.util.List;

import java.util.Set;

import java.util.stream.Collectors;

```
public Set<String> findDuplicates(List<String> list) {
    Set<String> seen = new HashSet<>();
    return list.stream()
        .filter(e -> !seen.add(e)) // add() returns false if element already exists
        .collect(Collectors.toSet());
}
```

- **Find a list of strings that end with 'X'.**

Java

```
public List<String> findStringsEndingWithX(List<String> strings) {
    return strings.stream()
        .filter(s -> s.endsWith("X"))
        .collect(Collectors.toList());
}
```

- **Convert Date to String.**

Java

```
import java.text.SimpleDateFormat;
import java.util.Date;
```

```
public String convertDateToString(Date date) {
    SimpleDateFormat formatter = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss");
    return formatter.format(date);
}
```

```
// Using Java 8 Date/Time API (preferred)
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
```

```
public String convertLocalDateToString(LocalDate date) {
    return date.format(DateTimeFormatter.ISO_DATE); // Or custom format
}
```

- **Get unique values/remove duplicates from a list (Java 8 stream vs. normal Java).**

Java

// Java 8 Stream

```
public <T> List<T> removeDuplicatesStream(List<T> list) {
    return list.stream().distinct().collect(Collectors.toList());
}
```

// Normal Java (using HashSet)

```
public <T> List<T> removeDuplicatesNormal(List<T> list) {
    return new java.util.ArrayList<>(new HashSet<>(list));
}
```

// To preserve order with normal Java:

// List<T> uniqueList = new ArrayList<>();

// Set<T> seen = new HashSet<>();

// for (T item : list) { if (seen.add(item)) { uniqueList.add(item); } }


```
// return uniqueList;
```

- **Convert Map value to List value.**

Java

```
import java.util.List;
```

```
import java.util.Map;
```

```
import java.util.stream.Collectors;
```

```
public <K, V> List<V> mapValuesToList(Map<K, V> map) {  
    return new java.util.ArrayList<>(map.values());  
    // Or using streams: map.values().stream().collect(Collectors.toList());  
}
```

- **Armstrong number using streams.**

Java

// An Armstrong number is a number that is the sum of its own digits each raised to the power of the number of digits.

// Example for 3-digit: $153 = 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$

```
public boolean isArmstrong(int number) {  
    String s = String.valueOf(number);  
    int n = s.length();  
    return number == s.chars()  
        .map(c -> Character.getNumericValue(c))  
        .mapToDouble(digit -> Math.pow(digit, n))  
        .sum();  
}
```

- **Basic Queue Implementation:** Implement a basic queue.

Java

```
import java.util.LinkedList; // Or ArrayDeque
```

```
import java.util.NoSuchElementException;
```

```
public class MyQueue<T> {  
    private LinkedList<T> list = new LinkedList<>();  
  
    public void enqueue(T item) {  
        list.addLast(item);  
    }  
  
    public T dequeue() {  
        if (isEmpty()) {  
            throw new NoSuchElementException("Queue is empty");  
        }  
    }  
}
```

```

        return list.removeFirst();
    }

    public T peek() {
        if (isEmpty()) {
            throw new NoSuchElementException("Queue is empty");
        }
        return list.getFirst();
    }

    public boolean isEmpty() {
        return list.isEmpty();
    }

    public int size() {
        return list.size();
    }
}

```

- **HashMap String Key Integer Value: Create a HashMap with String key and Integer value. Find even values and store the result in a HashMap.**

Java

```

import java.util.HashMap;
import java.util.Map;
import java.util.stream.Collectors;

```

```

public Map<String, Integer> filterEvenValues(Map<String, Integer> originalMap) {
    return originalMap.entrySet().stream()
        .filter(entry -> entry.getValue() != null && entry.getValue() % 2 == 0)
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
}

```

- **Scenario-based Code Output:** *(These require seeing the exact code. General approach is to trace execution, minding variable scopes, method calls, exception flow, and Java's evaluation order).*
 - **try-catch-finally with return statements:** finally block's return statement always overrides any return from try or catch blocks.
 - **List manipulation in methods:** Java passes objects by value (the reference is passed by value). So, if you modify the object (e.g., add to a list) inside the method, the original object outside will reflect changes. Reassigning the reference inside the method won't affect the original.
 - **Exception chaining:** Exceptions propagate up the call stack. A catch block can re-throw a new exception, linking it to the original cause (new

Exception(message, cause)).

- **Extract Last Digit:** Code in Java to extract the last digit from numbers using Stream API.

Java

```
import java.util.List;
```

```
import java.util.stream.Collectors;
```

```
public List<Integer> getLastDigits(List<Integer> numbers) {  
    return numbers.stream()  
        .map(n -> Math.abs(n % 10)) // Use Math.abs for negative numbers, % 10 gets  
last digit  
        .collect(Collectors.toList());  
}
```

- **Group Strings by Length:** Write a Java Stream API program to group elements of String type by their length.

Java

```
import java.util.List;
```

```
import java.util.Map;
```

```
import java.util.stream.Collectors;
```

```
public Map<Integer, List<String>> groupStringsByLength(List<String> strings) {  
    return strings.stream()  
        .collect(Collectors.groupingBy(String::length));  
}
```

- **Swap Two Variables: Swap two variables without a third variable.**

Java

```
public void swap(int a, int b) {  
    a = a + b; // a now holds sum of original a and b  
    b = a - b; // b now holds original a (sum - original b)  
    a = a - b; // a now holds original b (sum - original a)  
    // For numbers, also using XOR:  
    // a = a ^ b;  
    // b = a ^ b;  
    // a = a ^ b;  
}
```